

Q5) Git Branching and Merge Conflict Resolution

Create a Git repository named **Lab_Q5** and create three branches named **main**, **dev1**, and **dev2**. Create a text file named **Project.txt** containing 10 lines of content and commit it to the main branch. Modify the same line(s) of the file differently in both **dev1** and **dev2** branches and commit the changes in each branch. Attempt to merge **dev2** into **dev1**, resulting in a merge conflict. Resolve the conflict manually, verify the file contents, and complete the merge. Next, make additional modifications to the same line(s) in both **main** and **dev1** branches and commit the changes. Attempt to merge **dev1** into **main**, creating another merge conflict. Resolve the conflict manually, complete the merge, and verify that the final file contains the intended content from all branches. Finally, display the branch history and merge history to confirm successful conflict resolution and integration of all changes.

Step 1: Create a New Git Repository

We create a new Git repository named **Lab_Q5** to manage source code versions and track changes across multiple branches.

- `mkdir Lab_Q5` → Creates a new project directory.
- `cd Lab_Q5` → Moves into the project directory.
- `git init` → Initializes an empty Git repository.

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5
$ git init
Initialized empty Git repository in D:/learn/git/merge/Lab_Q5/.git/
```

Step 2: Create the Main Branch

We create the **main** branch which will act as the primary branch for the project.

Command

```
git checkout -b main
```

- **checkout** → Switches branches.
- **-b** → Creates a new branch.
- **main** → Name of the primary branch.

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main)  
$ git checkout -b main  
Switched to a new branch 'main'
```

Step 3: Create Project File and First Commit

We create a project file containing 10 lines of content and commit it as the initial project version.

Command

```
vim Project_<PRN>.txt
```

```
git add .
```

```
git commit -m "Initial project commit"
```

Explanation of attributes

- **vim** → Opens the file editor.
- **git add .** → Stages all changes.
- **git commit** → Creates a repository snapshot.
- **-m** → Specifies commit message.

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main)
$ vim Project_260240123060.txt

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main)
$ git add .
warning: in the working copy of 'Project_260240123060.txt',
1 be replaced by CRLF the next time Git touches it

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main)
$ git add .

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main)
$ git commit -m "Initial project commit"
[main (root-commit) 75af529] Initial project commit
1 file changed, 10 insertions(+)
create mode 100644 Project_260240123060.txt
```

Verify

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main)
$ cat Project_260240123060.txt
Line 1: Project Name - Git Branching Lab
Line 2: PRN - 260240123060
Line 3: Student Name - john j johnson jr
Line 4: Branch - Main
Line 5: This is the original project file.
Line 6: Version Control helps track changes.
Line 7: Git supports branching and merging.
Line 8: Merge conflicts occur when same lines are modified.
Line 9: Conflict resolution requires manual intervention.
Line 10: End of initial project content.

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main)
$ git branch
* main

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main)
$ git log
commit 75af529de38c0703b6e8c97f7cccd51e96b71bce (HEAD -> main)
Author: THE7MIST <asombale@gmail.com>
Date: Sat Jun 6 23:09:18 2026 +0530

    Initial project commit
```

Step 4: Create Development Branches

We create two independent development branches where different developers can work simultaneously.

Command

```
git checkout -b dev1
```

```
git checkout main
```

```
git checkout -b dev2
```

Explanation of attributes

- **dev1** → First developer branch.
- **dev2** → Second developer branch.
- **checkout -b** → Creates and switches to a new branch.

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main)
$ git checkout -b dev1
Switched to a new branch 'dev1'

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev1)
$ git checkout main
Switched to branch 'main'

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main)
$

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main)
$ git checkout -b dev2
Switched to a new branch 'dev2'
```

Verify

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev2)
$ git branch
dev1
* dev2
main
```

Step 5: Modify File in dev1 and Commit

We modify a specific line in the file on the **dev1** branch to create branch-specific changes.

Command

```
git checkout dev1
```

```
vim Project_<PRN>.txt
```

```
git add .
```

```
git commit -m "Modified content in dev1"
```

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev2)
$ git checkout dev1
Switched to branch 'dev1'

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev1)
$ vim Project_260240123060.txt
```

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev1)
$ git add .

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev1)
$ git commit -m "Modified content in dev1"
[dev1 b346a7e] Modified content in dev1
1 file changed, 5 insertions(+), 5 deletions(-)
```

Explanation of attributes

- **dev1** → Active development branch.
- Commit stores dev1-specific modifications.

Verify

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev1)
$ cat Project_260240123060.txt
Line 1: Project Name - Git Branching Lab
Line 2: PRN - 260240123060
Line 3: Student Name - john j johnson jr
Line 4: Branch - dev1
Line 5: dev1
Line 6: dev1
Line 7: dev1
Line 8: dev1
Line 9: Conflict resolution requires manual intervention.
Line 10: End of initial project content.

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev1)
$ git branch
* dev1
  dev2
  main

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev1)
$ git log --oneline
b346a7e (HEAD -> dev1) Modified content in dev1
75af529 (main, dev2) Initial project commit
```

Step 6: Modify Same Line in dev2 and Commit

We modify the same line differently on the **dev2** branch to intentionally create a merge conflict.

Command

```
git checkout dev2
```

```
vim Project_<PRN>.txt
```

```
git add .
```

```
git commit -m "Modified content in dev2"
```

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev1)
$ git checkout dev2
Switched to branch 'dev2'

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev2)
$ vim Project_260240123060.txt
```

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev2)
$ git add .

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev2)
$ git commit -m "Modified content in dev2"
[dev2 6f034ff] Modified content in dev2
1 file changed, 5 insertions(+), 5 deletions(-)
```

Explanation of attributes

- Different modification of the same line will trigger a conflict during merge.

Verify

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev2)
$ git commit -m "Modified content in dev2"
[dev2 6f034ff] Modified content in dev2
1 file changed, 5 insertions(+), 5 deletions(-)

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev2)
$ cat Project_260240123060.txt
Line 1: Project Name - Git Branching Lab
Line 2: PRN - 260240123060
Line 3: Student Name - john j johnson jr
Line 4: Branch - dev2
Line 5: dev2
Line 6: dev2
Line 7: dev2
Line 8: dev2
Line 9: Conflict resolution requires manual intervention.
Line 10: End of initial project content.

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev2)
$ git branch
dev1
* dev2
main

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev2)
$ git log --oneline
6f034ff (HEAD -> dev2) Modified content in dev2
75af529 (main) Initial project commit
```


Step 7: Merge dev2 into dev1

We attempt to merge changes from dev2 into dev1. Since both branches modified the same line differently, Git will detect a conflict.

Command

```
git checkout dev1
```

```
git merge dev2
```

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev2)
$ git checkout dev1
Switched to branch 'dev1'

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev1)
$ git branch
* dev1
  dev2
  main

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev1)
$ git merge dev2
Auto-merging Project_260240123060.txt
CONFLICT (content): Merge conflict in Project_260240123060.txt
Automatic merge failed; fix conflicts and then commit the result.
```

Explanation of attributes

- **merge** → Combines histories of two branches.
- **dev2** → Source branch to merge.

Verify

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev1|MERGING)
$ cat Project_260240123060.txt
Line 1: Project Name - Git Branching Lab
Line 2: PRN - 260240123060
Line 3: Student Name - john j johnson jr
<<<<<<< HEAD
Line 4: Branch - dev1
Line 5: dev1
Line 6: dev1
Line 7: dev1
Line 8: dev1
=====
Line 4: Branch - dev2
Line 5: dev2
Line 6: dev2
Line 7: dev2
Line 8: dev2
>>>>>>> dev2
Line 9: Conflict resolution requires manual intervention.
Line 10: End of initial project content.
```

Step 8: Resolve First Merge Conflict

What are we doing and why?

We manually resolve the conflict by editing the conflict markers and choosing the desired content.

Command

```
vim Project_<PRN>.txt
```

```
git add .
```

```
git commit -m "Resolved merge conflict between dev1 and dev2"
```

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev1|MERGING)
$ vim Project_260240123060.txt

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev1|MERGING)
$ git add .

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev1|MERGING)
$ git commit -m "Resolved merge conflict between dev1 and dev2"
[dev1 2848eda] Resolved merge conflict between dev1 and dev2
```

Explanation of attributes

- Conflict markers are removed manually.
- Commit finalizes the merge.

Step 9: Verify File Contents After Merge

We ensure the merged file contains the intended content.

Command

cat Project_<PRN>.txt

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev1)
$ cat Project_260240123060.txt
Line 1: Project Name - Git Branching Lab
Line 2: PRN - 260240123060
Line 3: Student Name - john j johnson jr
Line 4: Branch - dev1
Line 5: dev1
Line 6: dev1

Line 4: Branch - dev2
Line 5: dev2
Line 6: dev2
Line 7: dev2

Line 9: Conflict resolution requires manual intervention.
Line 10: End of initial project content.
```

Explanation of attributes

- `cat` → Displays file content.

Verify

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev1)
$ git log --oneline
2848eda (HEAD -> dev1) Resolved merge conflict between dev1 and dev2
6f034ff (dev2) Modified content in dev2
b346a7e Modified content in dev1
75af529 (main) Initial project commit

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev1)
$ git branch
* dev1
  dev2
  main
```

Step 10: Modify Same Line in Main Branch

We create another conflicting scenario by modifying the same line in the main branch.

Command

```
git checkout main
```

```
vim Project_<PRN>.txt
```

```
git add .
```

```
git commit -m "Updated content in main"
```

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev1)
$ git checkout main
Switched to branch 'main'

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main)
$ vim Project_260240123060.txt

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main)
$ git add .

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main)
$ git commit -m "Updated content in main"
[main f9c8aff] Updated content in main
1 file changed, 4 insertions(+), 4 deletions(-)

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main)
$ git branch
  dev1
  dev2
* main
```

Explanation of attributes

- Changes are committed to main branch.

Step 11: Modify Same Line in dev1 Branch

We make different modifications to the same line in dev1 branch.

Command

```
git checkout dev1
```

```
vi Project_<PRN>.txt
```

```
git add .
```

```
git commit -m "Updated content in dev1"
```

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main)
$ git checkout dev1
Switched to branch 'dev1'

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev1)
$ vim Project_260240123060.txt

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev1)
$ git add .

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev1)
$ git commit -m "Updated content in dev1"
[dev1 db69cc7] Updated content in dev1
1 file changed, 3 insertions(+)

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev1)
$ git branch
* dev1
  dev2
  main
```

Explanation of attributes

- Creates another merge conflict scenario.

Step 12: Merge dev1 into Main

We merge the development branch into main. Git detects another conflict because both branches modified the same line differently.

Command

```
git checkout main
```

```
git merge dev1
```

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (dev1)
$ git checkout main
Switched to branch 'main'

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main)
$ git branch
  dev1
  dev2
* main

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main)
$ git merge dev1
Auto-merging Project_260240123060.txt
CONFLICT (content): Merge conflict in Project_260240123060.txt
Automatic merge failed; fix conflicts and then commit the result.
```

Explanation of attributes

- Attempts integration of all development changes.

Verify

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main|MERGING)
$ cat Project_260240123060.txt
Line 1: Project Name - Git Branching Lab
Line 2: PRN - 260240123060
<<<<<<< HEAD
Line 3: Student Name - john j johnson jr
Line 4: Branch - Main
Line 5: main
Line 6: main
Line 7: main
Line 8: Main
=====
Line 3: Student Name - john j johnson jr
Line 4: Branch - dev1
Line 5: dev1
Line 6: dev1
what to do now After merger????
dev1
dev1

Line 4: Branch - dev2
Line 5: dev2
Line 6: dev2
Line 7: dev2

>>>>>>> dev1
Line 9: Conflict resolution requires manual intervention.
Line 10: End of initial project content.
```

Step 13: Resolve Second Merge Conflict

We manually resolve the second conflict and finalize integration.

Command

```
vim Project_<PRN>.txt
```

```
git add .
```

```
git commit -m "Resolved merge conflict between main and dev1"
```

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main|MERGING)
$ vim Project_260240123060.txt

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main|MERGING)
$ git add .

Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main|MERGING)
$ git commit -m "Resolved merge conflict between main and dev1"
[main 0679c68] Resolved merge conflict between main and dev1
```

Explanation of attributes

- Final conflict resolution commit.

Step 14: Verify Final File Content

What are we doing and why?

We confirm that all intended changes from all branches are present.

Command

cat Project_<PRN>.txt

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main)
$ cat Project_260240123060.txt
Line 1: Project Name - Git Branching Lab
Line 2: PRN - 260240123060
Line 3: Student Name - john j johnson jr
Line 4: Branch - Main
Line 5: main
Line 8: Main
Line 3: Student Name - john j johnson jr
Line 4: Branch - dev1
Line 5: dev1
Line 4: Branch - dev2
Line 5: dev2
Line 6: dev2
Line 7: dev2
Line 9: Conflict resolution requires manual intervention.
Line 10: End of initial project content.
```

Explanation of attributes

- Displays final integrated content.

Step 15: Display Branch History

We verify branch structure and successful merges.

Command

git branch

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main)
$ git branch
dev1
dev2
* main
```

Explanation of attributes

- Lists all available branches.
- Shows the current active branch.

```
$ git log
commit 0679c681217084dba63472285f24f04d744f201e (HEAD -> main)
Merge: f9c8aff db69cc7
Author: THE7MIST <asombale@gmail.com>
Date: Sat Jun 6 23:40:01 2026 +0530

    Resolved merge conflict between main and dev1

commit db69cc7f33f6832c19c51bd641b87232dcd6dcd9 (dev1)
Author: THE7MIST <asombale@gmail.com>
Date: Sat Jun 6 23:35:06 2026 +0530

    Updated content in dev1

commit f9c8aff28216d9615b156d814c5012120a47d550
Author: THE7MIST <asombale@gmail.com>
Date: Sat Jun 6 23:32:09 2026 +0530

    Updated content in main

commit 2848edab5d8e0f995e6f8e569a68fe5102e0a8d9
Merge: b346a7e 6f034ff
Author: THE7MIST <asombale@gmail.com>
Date: Sat Jun 6 23:27:09 2026 +0530

    Resolved merge conflict between dev1 and dev2

commit 6f034ff9ae6651d5dec8a47a880ed4feefd1d767 (dev2)
Author: THE7MIST <asombale@gmail.com>
Date: Sat Jun 6 23:20:55 2026 +0530

    Modified content in dev2

commit b346a7ec881de79f300b49d93019fc1be7cfd6a3
Author: THE7MIST <asombale@gmail.com>
Date: Sat Jun 6 23:16:34 2026 +0530

    Modified content in dev1

commit 75af529de38c0703b6e8c97f7cccd51e96b71bce
Author: THE7MIST <asombale@gmail.com>
Date: Sat Jun 6 23:09:18 2026 +0530

    Initial project commit
```

Step 16: Display Merge History

What are we doing and why?

We verify successful conflict resolution and integration history.

Command

git log --oneline --graph

```
Lenovo@theMist MINGW64 /d/learn/git/merge/Lab_Q5 (main)
$ git log --oneline --graph
* 0679c68 (HEAD -> main) Resolved merge conflict between main and dev1
|
* db69cc7 (dev1) Updated content in dev1
* 2848eda Resolved merge conflict between dev1 and dev2
|
| * 6f034ff (dev2) Modified content in dev2
| * | b346a7e Modified content in dev1
|/
* f9c8aff Updated content in main
|
* 75af529 Initial project commit
```

Explanation of attributes

- `log` → Displays commit history.
- `--oneline` → Compact output.
- `--graph` → Visual merge graph.
- `--all` → Shows history from all branches.

Memory Sheet (Command Flow)

`mkdir Lab_Q5`

`cd Lab_Q5`

`git init`

`git checkout -b main`

`vim Project_<PRN>.txt`

`git add .`

`git commit -m "Initial project commit"`

`git checkout -b dev1`

`git checkout main`

`git checkout -b dev2`

`git checkout dev1`

`vim Project_<PRN>.txt`

`git add .`

`git commit -m "Modified content in dev1"`

`git checkout dev2`

```
vim Project_<PRN>.txt
```

```
git add .
```

```
git commit -m "Modified content in dev2"
```

```
git checkout dev1
```

```
git merge dev2
```

```
vim Project_<PRN>.txt
```

```
git add .
```

```
git commit -m "Resolved merge conflict between dev1 and dev2"
```

```
git checkout main
```

```
vim Project_<PRN>.txt
```

```
git add .
```

```
git commit -m "Updated content in main"
```

```
git checkout dev1
```

```
vim Project_<PRN>.txt
```

```
git add .
```

```
git commit -m "Updated content in dev1"
```

```
git checkout main
```

```
git merge dev1
```

```
vim Project_<PRN>.txt
```

```
git add .
```

```
git commit -m "Resolved merge conflict between main and dev1"
```

```
cat Project_<PRN>.txt
```

```
git branch
```

```
git log --oneline --graph --all
```